

IncidentFlow

Setup and Verification Manual

A step-by-step guide to running the platform
and checking every part of it yourself.

Written for someone who has never seen this project before.
No prior knowledge of Laravel, React or Docker is assumed.

Project directory: `D:\IncidentFlow`

Contents

1	Before you start	3
1.1	What this document is	3
1.2	What is actually running	3
1.3	Two ways to run it	3
1.4	Opening a terminal	3
2	Path A — running it with Docker	4
2.1	Start everything	4
2.2	Confirm it came up	4
2.3	Open the application	4
3	Path B — running it manually	5
3.1	Step 1 — one-time preparation	5
3.2	Step 2 — start the API (terminal 1)	5
3.3	Step 3 — start the web app (terminal 2)	5
3.4	Step 4 — open it	6
3.5	Step 5 — optional: live updates	6
4	Checking each part is alive	7
4.1	The API	7
4.2	The realtime service	7
4.3	Verification checklist	7
5	The demo accounts	8
6	Guided tour	9
6.1	The incident register	9
6.2	Reporting an incident	9
6.3	Working the incident	10
6.4	Commanding the incident	10
6.5	Live updates	11
6.6	Resolving, and the postmortem	12
6.7	The metrics dashboard	12
6.8	Services	13
6.9	Exporting	13
6.10	Administration	13
7	Checking the promises yourself	15
7.1	A Viewer really is read-only	15
7.2	An incident cannot skip stages	15
7.3	A double-submitted report does not create two incidents	15
7.4	History cannot be rewritten	16
7.5	Repeated login attempts are throttled	16
8	Running the automated tests	17
8.1	The API	17
8.2	The realtime service	17
8.3	The web app	17
8.4	The browser tests	17
9	When something goes wrong	18
9.1	The page is blank or says it cannot load	18

9.2	“Address already in use”	18
9.3	The header says “Reconnecting...” forever	18
9.4	Every page says you are signed out	18
9.5	Starting completely fresh	18
9.6	Stopping everything	18
A	Command reference	19
B	Who can do what	19
C	The stages of an incident	19
D	Further reading	20

1. Before you start

1.1 What this document is

IncidentFlow is a platform for handling production incidents: staff report problems, engineers coordinate the response, and managers track how reliable the systems are over time.

This manual takes you from a cold machine to a running application, then walks you through every screen and asks you to confirm what you see. Every section ends with a checklist. If every box ticks, the system works.

1.2 What is actually running

Five separate programs cooperate. It helps to know what each one is for before you start them.

Program	Address	Its job
PostgreSQL	port 5432	Stores everything. The single source of truth.
Redis	port 6379	Carries live events and holds the job queue.
API (Laravel)	port 8000	All the rules. Decides who may do what.
Realtime (Node)	port 3001	Pushes live updates to open browsers.
Web app (React)	port 5173	What you look at.

The web app on its own does nothing useful — it asks the API for everything. If a screen is empty, the question is almost always “is the API running?” rather than “is the page broken?”

1.3 Two ways to run it

Path A — Docker

One command starts everything. Recommended if you have Docker Desktop installed. Skip to Section 2.

Path B — Manual

Start each program yourself. More steps, but you see exactly what is happening and it needs no Docker. Section 3.

To find out which you can use, open a terminal and type:

```
docker --version
```

If that prints a version number, use Path A. If it says the command is not recognised, use Path B.

1.4 Opening a terminal

Everything below is typed into a terminal.

- **Windows:** press the Start button, type `powershell`, press Enter.
- **macOS:** press `Cmd+Space`, type `terminal`, press Enter.

You will often need *several* terminals open at once, because some programs keep running and never give you the prompt back. Open a new one the same way; that is normal and expected.

2. Path A — running it with Docker

2.1 Start everything

```
cd D:\IncidentFlow
docker compose up --build
```

The first run takes several minutes because it downloads and builds everything. Later runs take seconds.

You should see: A long stream of coloured log lines that eventually settles down. Do *not* close this window — closing it stops the application.

2.2 Confirm it came up

Open a *second* terminal and run:

```
docker compose ps
```

You should see: A table listing postgres, redis, mailpit, api, realtime, web, nginx, horizon and scheduler. The important column is STATUS — everything should say **running** or **healthy**.

The **migrate** service is expected to say **exited (0)**. That is correct: it creates the database tables and loads the demo data, then stops. It is meant to run once, not forever.

2.3 Open the application

Go to <http://localhost:8080> in your browser.

Two other addresses are useful:

http://localhost:8025	Mailpit — every email the system sends lands here
http://localhost:8080/horizon	Horizon — the background job dashboard

No real email is ever sent in this setup. Mailpit catches everything, so you can read the notifications without anyone receiving them.

Now jump to Section 4.

3. Path B — running it manually

You will need four terminal windows. Keep them all open.

3.1 Step 1 — one-time preparation

Do this once, ever. Skip it on later runs.

```
cd D:\IncidentFlow\api
copy .env.local.example .env
composer install
php artisan key:generate
php artisan jwt:keys
php artisan migrate --seed
```

Taking those in turn:

```
copy .env.local.example .env
```

Creates the settings file. The `.env` file holds passwords and is deliberately never committed to version control.

```
composer install
```

Downloads the PHP libraries the API needs.

```
php artisan key:generate
```

Creates the encryption key for this installation.

```
php artisan jwt:keys
```

Creates the pair of security keys used to sign login tokens. This prints a long line starting `JWT_PUBLIC_KEY=` — you can ignore it for now.

```
php artisan migrate --seed
```

Builds the database tables and fills them with ninety days of realistic demo incidents.

You should see: The last command prints a table of demo accounts and the line *“17 incidents across 90 days, 3 still open.”*

Then prepare the web app, once:

```
cd D:\IncidentFlow\web
npm install
```

3.2 Step 2 — start the API (terminal 1)

```
cd D:\IncidentFlow\api
php artisan serve --port=8000
```

You should see: INFO Server running on [http://127.0.0.1:8000]. Leave this window open.

3.3 Step 3 — start the web app (terminal 2)

```
cd D:\IncidentFlow\web
npm run dev
```

You should see: A short box showing Local: `http://localhost:5173/`. Leave this window open.

3.4 Step 4 — open it

Go to `http://localhost:5173`.

Careful. At this point live updates are switched off. Everything else works — you just have to reload a page to see somebody else's changes. If that is enough for you, skip to Section 4. To turn live updates on, continue below.

3.5 Step 5 — optional: live updates

Live updates need Redis and the realtime service. Redis is already installed at `D:\redis`.

Terminal 3 — start Redis:

```
D:\redis\redis-server.exe D:\redis\incidentflow.conf
```

Check it answers, in any terminal:

```
D:\redis\redis-cli.exe ping
```

You should see: PONG

Now tell the API to use it. Open `D:\IncidentFlow\api\.env` in a text editor and change these lines (add them if missing):

```
REALTIME_ENABLED=true
REDIS_CLIENT=predis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
CACHE_STORE=redis
QUEUE_CONNECTION=redis
```

Stop the API (click its terminal, press `Ctrl+C`) and start it again.

Terminal 4 — start the realtime service. This is one long command; type it as a single line, or paste it.

```
cd D:\IncidentFlow\realtime
$env:PORT="3001"
$env:REDIS_URL="redis://127.0.0.1:6379"
$env:JWT_PUBLIC_KEY_PATH="D:/IncidentFlow/api/storage/keys/jwt-public.pem"
$env:JWT_ISSUER="incidentflow-api"
$env:JWT_AUDIENCE="incidentflow-realtime"
$env:CORS_ORIGINS="http://localhost:5173"
npx tsx src/index.ts
```

You should see: Two lines of JSON, the second containing `"message":"realtime service listening"`.

Terminal 5 — optional, the job worker. Without this, emails are prepared but never sent.

```
cd D:\IncidentFlow\api
php artisan queue:work redis
```

4. Checking each part is alive

Before touching the interface, confirm each piece is responding. Run these in a spare terminal.

4.1 The API

```
curl http://localhost:8000/api/v1/health/live
```

You should see: `{"status":"ok","service":"incidentflow-api",...}`

Now the deeper check, which tests the database and cache too:

```
curl http://localhost:8000/api/v1/health/ready
```

You should see: `"status":"ready"` and `"degraded":false`.

If you skipped the Redis step, this will say `"degraded":true`. That is not an error — it means “everything essential works, but live updates are unavailable.” The system is designed to keep running without Redis rather than fall over.

4.2 The realtime service

Only if you completed Step 5.

```
curl http://localhost:3001/readyz
```

You should see: `{"status":"ready","checks":{"redis":"ok",...}}`

4.3 Verification checklist

- ☐ The API answers `health/live` with `ok`.
- ☐ The API answers `health/ready` with `ready`.
- ☐ The web page loads and shows a sign-in form.
- ☐ (If applicable) Redis answers `PONG`.
- ☐ (If applicable) The realtime service answers `ready`.

5. The demo accounts

Six accounts exist. **The password for every one is incidentflow.**

They are not interchangeable — each sees a genuinely different application. That is the point of the next several sections.

Email	Role	Can do
admin@incidentflow.test	Administrator	Everything, plus the audit log
commander@incidentflow.test	Incident Commander	Assign people, change severity, publish postmortems, export
responder@incidentflow.test	Responder	Move an incident through its stages, post updates
responder2@incidentflow.test	Responder	The same, again — so there is somebody to assign
reporter@incidentflow.test	Reporter	Report incidents, comment
viewer@incidentflow.test	Viewer	Read only

Sign in and out often as you work through this manual. The quickest way to understand the permission system is to look at the same screen as two different people.

6. Guided tour

Work through these in order. Each has instructions, then a checklist.

6.1 The incident register

Sign in as `reporter@incidentflow.test`.

You land on a table of incidents. Things to try:

1. Type **checkout** into the search box. The list narrows as you type.
2. Set **All severities** to **SEV1**. Only the most serious incidents remain.
3. Tick **Active only**. Anything already resolved disappears.
4. Clear the filters again.

Look at the **Running / resolved in** column. For open incidents this is a live clock showing how long the problem has been going on.

Look at the navigation bar. Count the tabs — there should be **four**.

- ☐ 17 incidents are listed.
- ☐ Searching narrows the list.
- ☐ Filtering by severity works.
- ☐ There are four navigation tabs, and *no* Admin tab.
- ☐ A **Report incident** button is visible.

The missing Admin tab is deliberate. A Reporter has no permission to read the audit log, so the tab does not exist for them — it is not merely greyed out.

6.2 Reporting an incident

Still signed in as the Reporter, click **Report incident**.

Fill it in:

- **What is happening?** — *Card payments failing for UK customers*
- **Severity** — SEV-1
- **Service** — Checkout API
- **What do we know?** — anything you like

Read the amber warning before you submit. It tells you what choosing SEV-1 actually causes. Then click **Report incident**.

You are taken straight to the new incident.

- ☐ You are moved to the new incident's own page.
- ☐ It has a reference like INC-0018.
- ☐ A red **Open** badge and a **Postmortem required** badge appear.

- ☐ The timeline already has one entry: “... *reported the incident*”.
- ☐ There are **no** buttons to change the status.

Careful. That last point is the important one. A Reporter cannot move an incident forward. Try to find a way to do it — you should not be able to.

Check the emails went out

If you are on Path A (Docker), open `http://localhost:8025`.

You should see: Four emails, one to each person who can be paged. There is *nothing* addressed to the Reporter or the Viewer — neither can be assigned to an incident, so neither is on the on-call rota.

6.3 Working the incident

Sign out. Sign in as `responder@incidentflow.test`. Open the incident you just created.

Notice you now have buttons. Count them — there are **two**: **Mark acknowledged** and **Mark resolved**.

There is no “Mark closed” button. An incident cannot be closed before it has been resolved, so that option is not offered. The buttons on screen are generated from the rules, so you can never be shown an action that would be refused.

Click **Mark acknowledged**. Watch the buttons change to **Mark mitigated** and **Mark resolved**.

Now scroll down and try the two writing areas:

- **Post an update** — the public record of what is happening. Tick *Safe for a customer status page* before posting one.
 - **Discussion** — internal chat between responders.
- ☐ Exactly two status buttons appear before you click.
 - ☐ After acknowledging, the badge turns amber and reads **Acknowledged**.
 - ☐ The buttons change to Mitigated / Resolved.
 - ☐ The timeline gains “... *acknowledged the incident*”.
 - ☐ A posted update shows a green **Public** tag when you tick the box.
 - ☐ Your comment appears in Discussion immediately.
 - ☐ The right-hand panel shows **Time to acknowledge** with a target.

6.4 Commanding the incident

Sign out. Sign in as `commander@incidentflow.test`. Open the same incident and scroll to the right-hand column.

You now have two panels the Responder did not: **Responders** and **Change severity**.

Assigning someone

Open the **Assign someone...** dropdown and read the names.

You should see: Only people who can actually be paged. Ahmed Hassan (Reporter) and Sofia Rossi (Viewer) are absent — assigning a Viewer would look like coverage while providing none.

Pick someone. They appear in the list with a button to remove them again.

Changing severity

This one is worth doing carefully, because the behaviour is deliberately asymmetric.

1. Choose a *higher* severity (e.g. SEV-1 to SEV-2 is lower; go the other way). No extra field appears — making something more serious needs no justification.
2. Now choose a *lower* severity, such as SEV-4. A **reason** box appears, and it is required.

Quietly downgrading an incident changes whether a postmortem is required and how it appears in every later report. So a downgrade goes on the record; an escalation does not need to.

- ☐ The Responders and Change severity panels are visible.
- ☐ The assignment dropdown excludes the Reporter and the Viewer.
- ☐ An assigned person appears in the list with a remove button.
- ☐ Raising severity asks for no reason.
- ☐ Lowering severity demands a written reason.

6.5 Live updates

Careful. This section only works if you completed Path A, or Path B Step 5. Look at the top right of the screen: if it says a green **Live**, you are ready. If it says amber **Reconnecting...**, skip this section.

This is best seen with two browser windows side by side.

1. In your normal window, stay signed in as the Commander, on an incident.
2. Open a **private** / **incognito** window (Ctrl+Shift+N in Chrome). Go to the same address and sign in as `responder@incidentflow.test`. Open the *same* incident.
3. Arrange the windows so you can see both.
4. In the **private** window, post a comment.
5. Watch the **other** window without touching it.

You should see: The comment and a new timeline entry appear in the first window within a second or two. You did not reload anything.

- ☐ The header shows a green **Live** badge.
- ☐ A change made in one window appears in the other unprompted.
- ☐ The incident list also updates itself the same way.

6.6 Resolving, and the postmortem

As the Commander, click **Mark resolved**.

Notice the buttons become **Mark closed** and **Mark acknowledged**. Resolved is not the end — if the problem returns you reopen it, and the system discards the old resolution time rather than flattering the statistics.

Now scroll to the **Postmortem** section.

1. Read the four required sections. Each is labelled *required* — *blocks publishing* in amber, and each has a hint telling you what belongs there.
2. Fill in **Summary** only, then click **Start postmortem**.
3. Look at the **Publish** button — it is greyed out, and a note below lists exactly what is still missing.
4. Now fill in Root cause, Impact and Resolution. Watch each label change from amber *required* to green *complete*.
5. Click **Save draft**, then **Publish**.

You should see: A green banner naming who published it and when. Every box becomes read-only, and the Save and Publish buttons disappear entirely.

Now try to edit it. You cannot — and neither can an administrator.

- ☐ Four sections are marked as required.
- ☐ Publish is disabled while any of them is empty.
- ☐ The message below names precisely which are missing.
- ☐ Labels turn green as you complete each one.
- ☐ Publish becomes available only when all four are filled.
- ☐ After publishing, every field is locked and the buttons vanish.

6.7 The metrics dashboard

Click **Metrics**. Switch between the 7, 30 and 90 day buttons.

Read the two large numbers carefully. Under **Mean time to acknowledge** there is a smaller line reading something like *p50 9m · p95 5h*.

This is the most important idea on the page. “p50” is the middle case: half of all incidents were picked up faster than that. “p95” is the bad case: one in twenty took at least that long. An average on its own hides both. If the average is 46 minutes but the middle case is 9 minutes, then most incidents are handled quickly and a few are handled very slowly — and those are two completely different problems to fix.

Scroll down for the SLA table and the two charts. Hover over a bar in **Opened vs resolved** to see that day’s figures.

- ☐ Four summary figures appear at the top.
- ☐ Each average is accompanied by p50 and p95.

- ☐ The SLA table shows a target and an attainment per severity.
- ☐ Both charts draw, and the bar chart responds to hovering.
- ☐ Changing the period changes the numbers.

6.8 Services

Click **Services**. These are the systems that can break.

- ☐ Five services are listed.
- ☐ Each shows a tier and the team that owns it.
- ☐ Services with live incidents show a red count.

6.9 Exporting

Still as the Commander, go back to **Incidents**. There is now an **Export CSV** button.

Do this in order, because the point is what the file contains:

1. Without any filters, click **Export CSV**. Open the downloaded file in Excel. Count the rows.
2. Back in the browser, tick **Active only**. Note how many rows the table now shows.
3. Click **Export CSV** again and open that file.

You should see: The second file contains exactly the rows that were on screen — not everything. The export always matches what you were looking at.

- ☐ A file downloads, named after the organisation and the date.
- ☐ It opens in Excel with readable column headings.
- ☐ Filtering the screen changes what the file contains.

6.10 Administration

Sign out. Sign in as `admin@incidentflow.test`.

Count the navigation tabs — there are now **five**. The **Admin** tab has appeared.

Open it. The top half lists everyone and their role. Scroll down for the audit log.

You should see: Every action you have taken during this tour, most recent first, with who did it and when. Underneath each is a long identifier — that is the tracking number for that request, shared across all parts of the system.

- ☐ Five navigation tabs, including Admin.
- ☐ All six members are listed with their roles.
- ☐ The audit log shows your actions from this session.
- ☐ Each entry names a person.
- ☐ Your CSV exports appear in the log too.

Exports are recorded because taking customer data out of the system is itself something worth being able to account for later.

7. Checking the promises yourself

The sections above show the product working. These show it refusing to misbehave. Each is a claim you can test.

7.1 A Viewer really is read-only

Sign in as `viewer@incidentflow.test`.

- ☐ No **Report incident** button anywhere.
- ☐ Opening an incident offers no status buttons.
- ☐ No comment box.
- ☐ No Admin tab.

Now try to get around the interface. In the address bar, type `http://localhost:5173/admin` directly.

You should see: You are not shown the audit log. The restriction lives in the server, not in whether a button was drawn.

7.2 An incident cannot skip stages

The interface never offers an illegal step, so this must be tested directly against the API.

First get a login token (this is one long line):

```
curl -X POST http://localhost:8000/api/v1/auth/login -H "Content-Type: application/json" -d '{"email":"responder@incidentflow.test","password":"incidentflow"}'
```

Copy the long string after `"access_token":`. Now try to close an incident that is merely open — replace `PASTE_TOKEN` with it, and 18 with any open incident’s number:

```
curl -X POST http://localhost:8000/api/v1/incidents/18/status -H "Authorization: Bearer PASTE_TOKEN" -H "X-Organization: northwind" -H "Content-Type: application/json" -d '{"status":"closed"}'
```

You should see: A refusal containing `"code":"incident.illegal_transition"`, and helpfully a list of which steps *are* allowed from here.

7.3 A double-submitted report does not create two incidents

If someone taps “Report” twice on a bad connection, only one incident should exist. Run this command *twice*, unchanged:

```
curl -X POST http://localhost:8000/api/v1/incidents -H "Authorization: Bearer PASTE_TOKEN" -H "X-Organization: northwind" -H "Content-Type: application/json" -H "Idempotency-Key: my-test-key-123" -d '{"title":"Testing duplicate submission","severity":"sev3"}'
```

You should see: Both responses describe the *same* incident, with the same reference number. Check the incident list — there is only one.

The **Idempotency-Key** is what makes this safe. The application generates one automatically for every report you file through the interface.

7.4 History cannot be rewritten

Look at any incident's timeline. There is no edit button and no delete button — not for a Responder, not for a Commander, not for an Administrator.

This is enforced in three separate places, so that removing one would not be enough. The reason is simple: a postmortem is written from the timeline, so if the timeline could be altered afterwards, nothing concluded from it could be trusted.

- ☐ No timeline entry can be edited or deleted by anyone.
- ☐ No audit log entry can be edited or deleted by anyone.
- ☐ A published postmortem cannot be edited, even by its author.

7.5 Repeated login attempts are throttled

Run this six times quickly, with the wrong password on purpose:

```
curl -X POST http://localhost:8000/api/v1/auth/login -H "Content-Type: application/json" -d "{\"email\":\"admin@incidentflow.test\",\"password\":\"wrong\"}"
```

You should see: The first few refuse politely. Around the sixth, you get `"code":"rate_limited"` and a `Retry-After` header.

Also notice: guessing an address that does not exist gives *exactly* the same refusal as a wrong password. The login form cannot be used to discover who has an account.

8. Running the automated tests

You do not need the application running for these. Each is independent.

8.1 The API

```
cd D:\IncidentFlow\api
php vendor/bin/phpunit
```

You should see: OK (104 tests, 518 assertions) in green, after a few seconds.

8.2 The realtime service

```
cd D:\IncidentFlow\realtime
npm test
```

You should see: Tests 26 passed (26)

8.3 The web app

```
cd D:\IncidentFlow\web
npm test
```

You should see: Tests 23 passed (23)

8.4 The browser tests

These drive a real browser through the whole application, so the system must be running first.

```
cd D:\IncidentFlow\web
npx playwright install chromium
npm run e2e
```

You should see: 7 passed, taking a couple of minutes.

If you are on Path B rather than Docker, add `E2E_EXPECT_TIMEOUT=45000` before the command. The development server answers one request at a time, so pages load more slowly than the tests expect by default.

- ☐ API: 104 tests pass.
- ☐ Realtime: 26 tests pass.
- ☐ Web: 23 tests pass.
- ☐ Browser: 7 tests pass.

9. When something goes wrong

9.1 The page is blank or says it cannot load

The API is almost certainly not running. Check its terminal window, then:

```
curl http://localhost:8000/api/v1/health/live
```

If that fails, restart the API.

9.2 “Address already in use”

Something is already using that port — often a copy of this application you forgot to stop. On Windows:

```
netstat -ano | findstr :8000
taskkill /F /PID <the number from the last column>
```

9.3 The header says “Reconnecting...” forever

Live updates are unavailable. In order, check:

1. Is Redis running? `D:\redis\redis-cli.exe ping`
2. Is the realtime service running? `curl http://localhost:3001/readyz`
3. Does the API's `.env` say `REALTIME_ENABLED=true`?
4. Did you restart the API after changing that?

Careful. This never stops the rest of the application working. Everything except automatic updating continues normally — you just reload to see changes.

9.4 Every page says you are signed out

The security keys are missing or were regenerated. Rebuild them:

```
cd D:\IncidentFlow\api
php artisan jwt:keys --force
```

Then restart the API and the realtime service, and sign in again.

9.5 Starting completely fresh

This deletes all data and rebuilds the demo from scratch. It is safe — there is nothing real in here.

```
cd D:\IncidentFlow\api
php artisan migrate:fresh --seed
```

9.6 Stopping everything

Docker: press `Ctrl+C` in its window, then `docker compose down`.

Manual: press `Ctrl+C` in each terminal window.

A. Command reference

Command	What it does
<code>docker compose up --build</code>	Start everything (Path A)
<code>docker compose down</code>	Stop everything
<code>docker compose logs -f api</code>	Watch the API's log
<code>php artisan serve --port=8000</code>	Start the API (Path B)
<code>php artisan queue:work redis</code>	Start the job worker
<code>php artisan migrate --seed</code>	Create tables, load demo data
<code>php artisan migrate:fresh --seed</code>	Wipe and rebuild everything
<code>php artisan jwt:keys --force</code>	Regenerate the security keys
<code>php artisan incidentflow:prune --dry-run</code>	Show what housekeeping would delete
<code>npm run dev</code>	Start the web app
<code>npm test</code>	Run that project's tests
<code>npm run e2e</code>	Run the browser tests

B. Who can do what

	<i>Viewer</i>	<i>Reporter</i>	<i>Responder</i>	<i>Commander</i>	<i>Admin</i>
View incidents	✓	✓	✓	✓	✓
View metrics	✓	✓	✓	✓	✓
Report an incident		✓	✓	✓	✓
Comment		✓	✓	✓	✓
Change status			✓	✓	✓
Post updates			✓	✓	✓
Assign responders				✓	✓
Change severity				✓	✓
Publish postmortems				✓	✓
Export to CSV				✓	✓
Manage services					✓
Manage members					✓
Read the audit log					✓
Delete incidents					✓

Each role includes everything the one before it can do.

C. The stages of an incident

Stage	Means	Can move to
Open	Reported, nobody has responded	Acknowledged, Resolved
Acknowledged	Somebody is working on it	Mitigated, Resolved
Mitigated	Under control, not fixed	Resolved, Acknowledged
Resolved	Fixed	Closed, Acknowledged
Closed	Finished	— nothing

Two rules worth knowing:

- Nothing ever returns to **Open**. Once a person has seen it, the “nobody has looked at this” state would no longer be true.
- **Resolved** can go back to **Acknowledged** if the problem returns. When that happens the recorded resolution time is discarded — an incident that came back was never really resolved.

D. Further reading

README.md	Overview and quick start
docs/architecture.md	Why the system is built this way
docs/api.md	Every endpoint, in detail
docs/openapi.yaml	The machine-readable API specification
docs/runbook.md	Deploying it, and what to do when it breaks
docs/interview-notes.md	The design decisions, argued in depth

If every checkbox in this manual ticks, the system is working as designed.